



Duale Hochschule Baden-Württemberg Mosbach

Projekt Webprogrammierung

Kapitelweise Portfolio

Verfasser:

Fabrice Böhme (1523396), Kira Kappes (1768280),

Elias Langhans (7598556), Niklas Ültzhöfer (8485275)

Studiengang:

Wirtschaftsinformatik

Kurs:

MOS-WWI25A

Studiengangsleitung:

Prof. Dr. Christoph Sturm

INHALTSVERZEICHNIS

1 Abstract 5

1.1 Deutsch 5

1.2 Englisch 6

2 Einführung in das Projekt 7

2.1 Motivation 7

2.2 Was leistet der Service? 7

2.3 Architektur im Überblick 9

3 API-Design 11

4 Implementierung - unser Lern- und Projektverlauf 15

4.1 Kapitelweise-Auth 16

4.2 Kapitelweise-Backend 19

4.3 Kapitelweise-GraphQL 21

4.4 Kapitelweise-OpenLibrary 23

4.5 Kapitelweise-Frontend 24

5 Wissenschaftliche Reflexion der Teammitglieder 25

6 Quellenverzeichnis 33

ABBILDUNGSVERZEICHNIS

Abbildung 1 – Click-Flow Login und Registrierung 7
Abbildung 2 – Click-Flow Bücher suchen, vorschlagen und Buchclub verlassen 8
Abbildung 3 – Click-Flow Buch auswählen, abschließen und Mitglieder einladen (Admin) 8
Abbildung 4 – Click-Flow Meeting erstellen und verwalten. 9
Abbildung 5 – Architekturdiagramm Kapitelweise – Darstellung der Queues. (KI-generiert). 10
Abbildung 6 – Beispiel grundlegender Aufbau 16
Abbildung 7 – JWT-Service 17
Abbildung 8 – Spring Secure Filter-Chain 17
Abbildung 9 – Cookie Einbindung 18
Abbildung 10 – Probleme mit dem Cookie 18
Abbildung 11 - Erstellung in der application.yml 19
Abbildung 12 – Vorschlagsverarbeitung – Direkte Antwort 20
Abbildung 13 - Vorschlagsverarbeitung – Event an die Queue 20
Abbildung 14 - Vorschlagsverarbeitung – Verarbeitung der Antwort 20
Abbildung 15 – GraphQL Implementierung 22
Abbildung 16 – GraphQL SchemaMapping 22
Abbildung 17 – OpenLibrary Antwort vorher 23
Abbildung 18 – OpenLibrary Antwort nachher 23
Abbildung 19 – OpenLibrary erneute Abfrage 24

TABELLENVERZEICHNIS

Tabelle 1 – Übersicht der Aufgaben und Technologien 10
Tabelle 2 – Übersicht Statuscodes 11
Tabelle 3 – API Auth-Service 11
Tabelle 4 – API GraphQL 12
Tabelle 5 – API Backend-Service 14
Tabelle 6 – API OpenLibrary-Service 14
Tabelle 7 – Übersicht der Sprintwochen 15

1 Abstract

1.1 Deutsch

Kapitelweise ist ein Webservice, mit dem Buchclubs ihren wiederkehrenden Ablauf aus Buch Vorschlagen, Buch Auswählen, Lesen und Treffen an einem Ort organisieren, statt ihn in WhatsApp-Gruppen zu verlieren. In sechs Sprints ist daraus ein System aus einem Frontend, vier Backend-Services und einem Alexa-Skill entstanden. Keiner von uns hatte vorher ein Backend dieser Art gebaut. Die nötigen Skills haben wir uns daher während des Projekts erarbeitet, mit Tutorials und Recherche.

Das Portfolio zeichnet nach, wie sich ein Monolith schrittweise zu dieser Architektur entwickelt hat. Erst die Trennung danach, welchem Service welche Daten gehören, hat eigene Datenbanken pro Service ermöglicht. Diese Erkenntnis kam nicht am Anfang, sondern erst beim Umbau, und sie hat uns danach bei weiteren Entscheidungen geleitet. Als unsere stärkste Entscheidung sehen wir die Prüfung der Buchvorschläge im Hintergrund. Sie war die komplizierteste Umsetzung, aber auch die am gründlichsten geplante, weil wir den Ablauf vorher gemeinsam aufgezeichnet haben. Nutzer bekommen dadurch sofort eine Antwort und sehen später, ob ihr Vorschlag angenommen wurde. Bei einem Code-Review fiel uns außerdem auf, dass Vorschläge für immer hängen bleiben konnten. Das hätten wir sonst nicht gefunden.

Am meisten gelernt haben wir aus den Rückschlägen. Zweimal hatten wir eine Lösung lokal fertig gebaut und merkten erst beim Deployment, dass sie in der Zielumgebung nicht funktioniert. Bei der Alexa-Anbindung hätten wir die Voraussetzungen des externen Dienstes ebenfalls früher prüfen müssen. Bewährt haben sich eine API-Dokumentation vor der Implementierung, das gemeinsame Aufzeichnen komplexer Abläufe und regelmäßige Code-Reviews. Beim nächsten Mal würden wir früh und häufig deployen, vor dem ersten Code festlegen, welchem Service welche Daten gehören, und gemeinsame Konventionen von Beginn an einführen. Gefundene Fehler würden wir außerdem gleich in den anderen Services mitprüfen.

1.2 Englisch

Kapitelweise is a web service that lets book clubs organize their recurring routine of suggesting books, choosing a book, reading, and meeting in one place, instead of losing it in WhatsApp groups. Over six sprints, it grew into a system made up of a frontend, four backend services, and an Alexa skill. None of us had built a backend of this kind before. We therefore acquired the necessary skills during the project, through tutorials and research.

The portfolio traces how a monolith evolved step by step into this architecture. Only once we split the system by which service owns which data could each service get its own database. This insight did not come at the start but only during the restructuring, and it guided us in later decisions. We consider the background check of book suggestions our strongest decision. It was the most complicated implementation but also the most thoroughly planned, because we mapped out the flow together beforehand. As a result, users get an immediate response and see later whether their suggestion was accepted. During a code review, we also noticed that suggestions could get stuck forever. We would not have found that otherwise.

We learned the most from our setbacks. Twice we had finished a solution locally and only discovered during deployment that it did not work in the target environment. With the Alexa integration, we likewise should have checked the requirements of the external service earlier. What proved worthwhile were API documentation before implementation, mapping out complex flows together, and regular code reviews. Next time, we would deploy early and often, decide before the first line of code which service owns which data, and introduce shared conventions from the very start. We would also check any bug we find in one service right away in the other services.

2 Einführung in das Projekt

2.1 Motivation

Buchclubs organisieren sich heute meist über WhatsApp-Gruppen. Das funktioniert nur so lange, bis man zum ersten Mal weit nach oben scrollen muss, um das aktuelle Buch oder den Termin des nächsten Treffens wiederzufinden. Die Idee für *Kapitelweise* stammt aus unserem direkten Umfeld: Die Mütter von zwei Teammitgliedern sind selbst in Buchclubs, die sich ausschließlich über WhatsApp organisieren.

Auf diesen Erfahrungen soll unser Webservice aufbauen. Wir wollen Buchclubs eine klare Organisation und Verwaltung bieten. Alle wichtigen Informationen sind dabei an einem Ort gebündelt, statt in einem endlosen Chatverlauf unterzugehen.

2.2 Was leistet der Service?

Kapitelweise bietet folgende Funktionen:

- Registrierung, Login und Logout.
- Buchclubs gründen und andere Nutzer einladen (Admin).
- Einladungen annehmen oder ablehnen.
- Buchclub verlassen (Member).
- Bücher über die Open Library suchen und vorschlagen.
- Ein Buch aus den Vorschlägen auswählen und nach dem Lesen abschließen (Admin).
- Treffen anlegen (Admin) und zu- oder absagen.
- Aktuelles Buch und Vorschläge per Alexa abfragen.
- Startseite mit Clubs und Einladungen auch offline nutzbar.

Die einzelnen Funktionen werden im Folgenden über Happy Path Click-Flows vorgestellt:

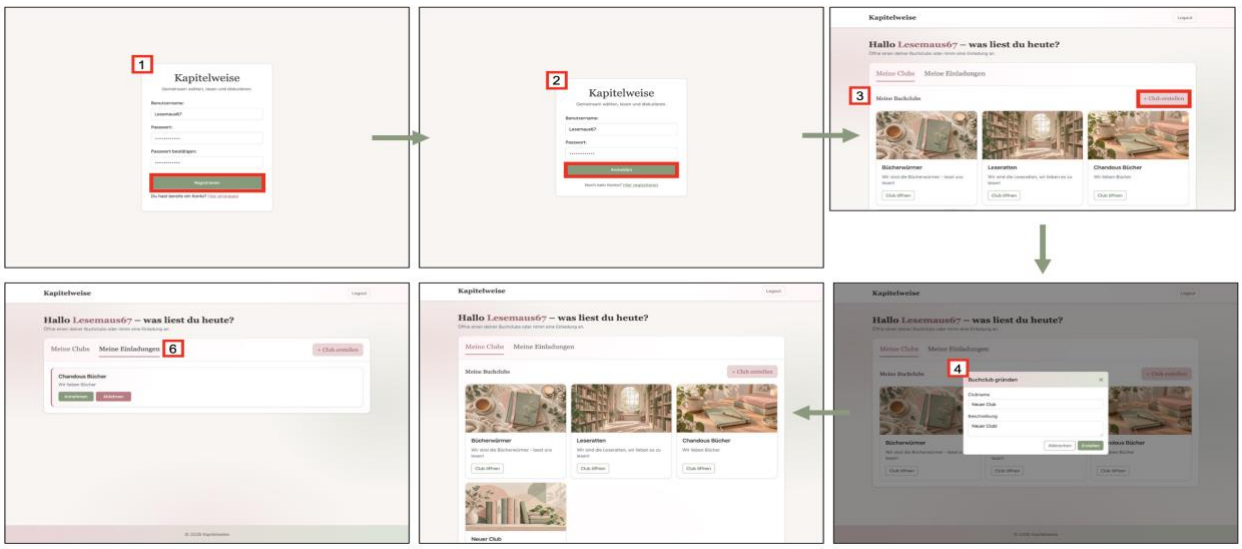


Abbildung 1 – Click-Flow Login und Registrierung

Nach erfolgreicher Registrierung landet der User erneut auf der Login-Seite und gibt dort seine Login-Daten ein. Danach landet er auf der Übersichtsseite und sieht, falls vorhanden, seine Buchclubs, in denen er Mitglied ist. Von dort aus kann er über „Buchclub gründen“ auch einen neuen Buchclub anlegen und sieht diesen anschließend in seiner Übersichtsseite (Schritt 1-5). Zusätzlich sieht der User unter „Einladungen“ auch alle seine aktiven Einladungen zu einem Buchclub (Schritt 6). Nach Annahme sieht er den Buchclub in seiner Übersicht und kann diesen öffnen. Mit Öffnen eines Buchclubs gelangt man auf dessen Übersichtsseite.

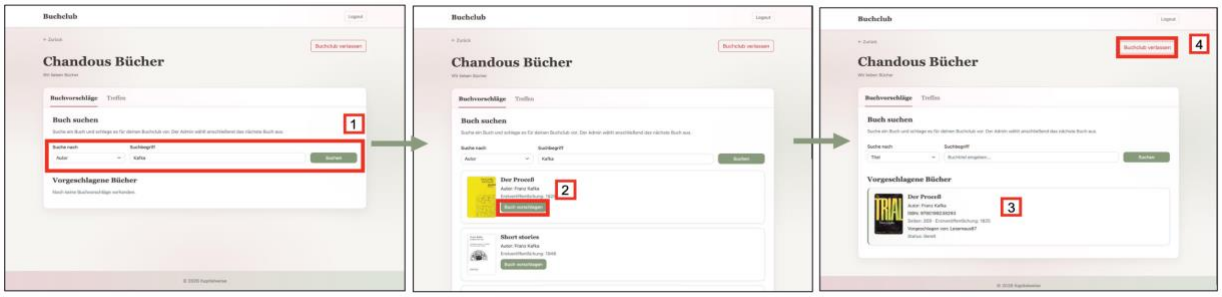


Abbildung 2 – Click-Flow Bücher suchen, vorschlagen und Buchclub verlassen

Dort kann er unter Buchvorschläge alle aktiven Buchvorschläge sehen und selbst welche vorschlagen. Über die Buchsuche kann er Titel, ISBN und Autor suchen (Schritt 1). Nach Auswahl eines Buchs erscheint es unter Buchvorschläge (Schritt 2, 3). Ein Mitglied kann außerdem ein Buchclub verlassen (Schritt 4).

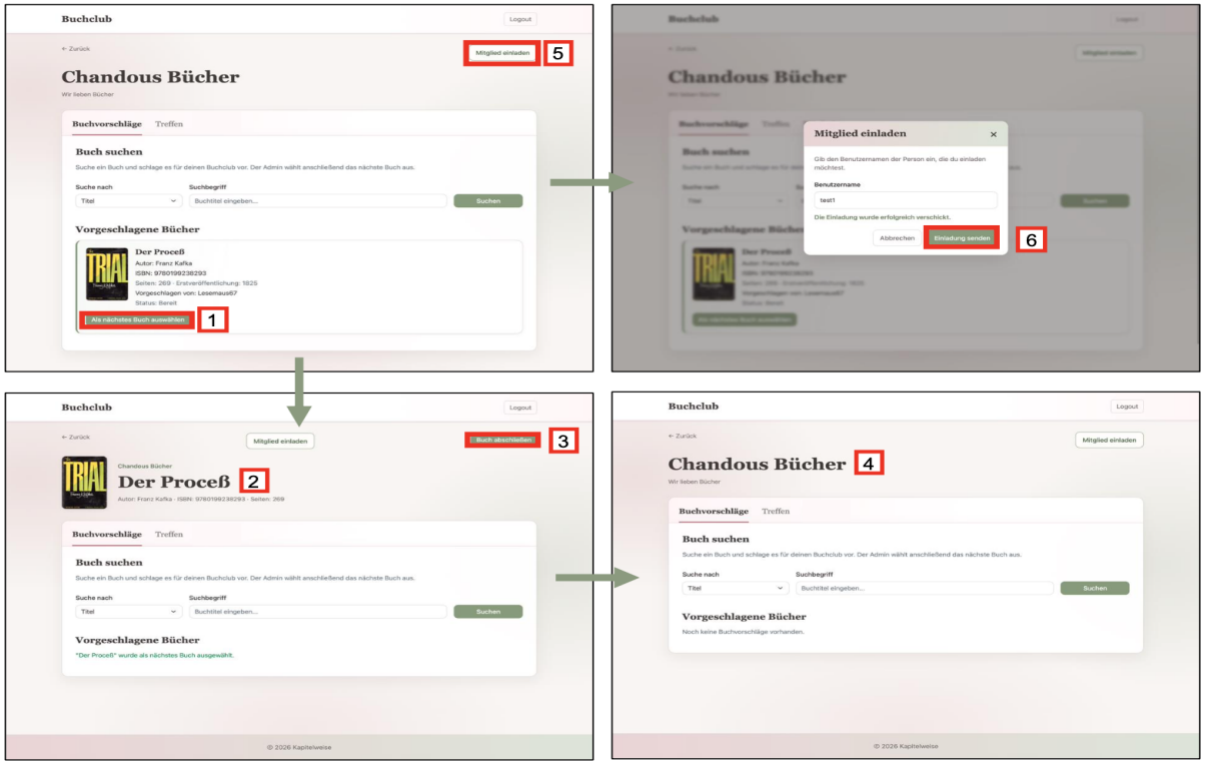


Abbildung 3 – Click-Flow Buch auswählen, abschließen und Mitglieder einladen (Admin)

Der Admin eines Buchclubs kann das einen Vorschlag als aktuelles Buch auswählen (Schritt 1). Es erscheint dann als aktuelles Buch auf der Seite (Schritt 2). Zusätzlich kann er es danach abschließen (Schritt 3, 4).

Außerdem kann der Admin neue Mitglieder über einen Usernamen einladen (Schritt 5, 6).

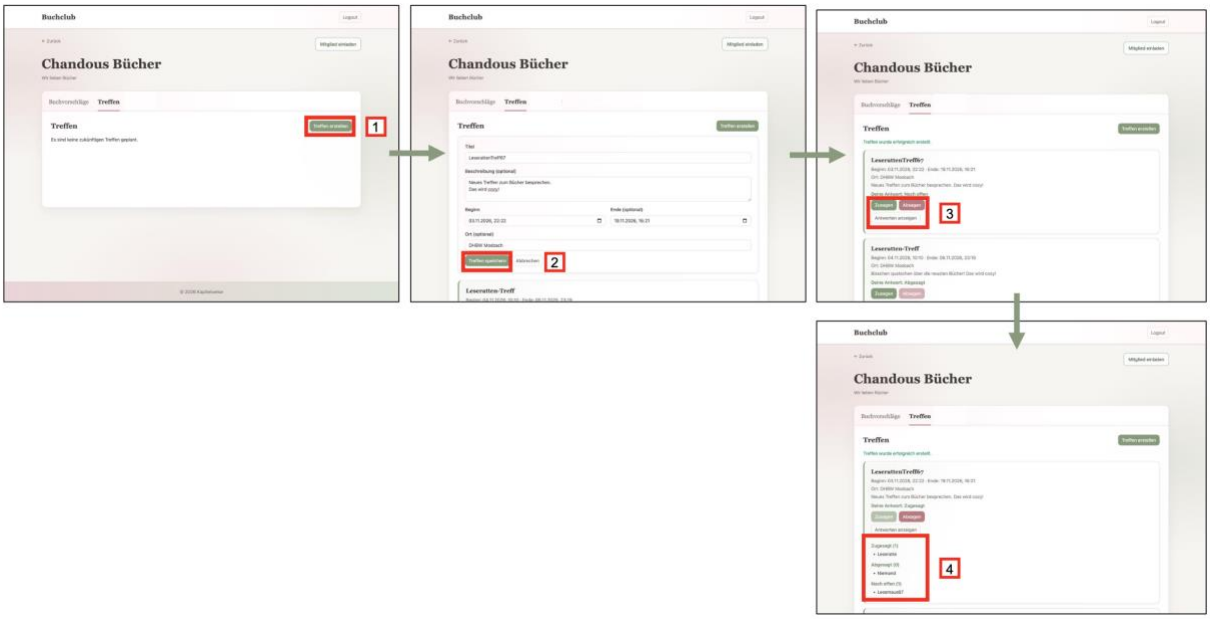


Abbildung 4 – Click-Flow Meeting erstellen und verwalten.

Abschließend kann der Admin unter dem Punkt „Treffen“ ein neues Treffen anlegen. Dies sehen dann alle Mitglieder und können zu- oder absagen. Der Admin sieht außerdem, wer bisher alles zugesagt hat.

2.3 Architektur im Überblick

Der Webservice besteht insgesamt aus vier Backend- und einem Frontend-Service. Zusammenfassend lassen sich die Aufgaben der einzelnen Repositories folgendermaßen definieren:

Repository	Technik	Aufgabe
kapitelweise-frontend	HTML, CSS, JavaScript, Bootstrap 5, jQuery, Angular	Login, Registrierung (MPA), Startseite (Angular/SPA), Buchclub-Seite (MPA)
kapitelweise-auth	Spring Boot, Spring Security, JDBC, JWT, AWS SQS	/register und /login, erzeugt das JWT, meldet neue User über SQS
kapitelweise-graphql	Spring for GraphQL, Spring Security, JDBC, JWT, AWA SQS	Startseite (eigene Clubs, Einladungen, Club erstellen) sowie Club-Details und Einladen auf Buchclub-Seite
kapitelweise-backend	Spring Boot, Spring Security, JDBC, JWT, AWS SQS	REST-API der Buchclub Seite: Buchvorschläge, aktuelles Buch, Treffen mit Zu-Absage, Alexa

Repository	Technik	Aufgabe
Kapitelweise-OpenLibrary	Spring Boot, Spring Security, JDBC, AWA SQS	Buchsuche über die Open Library API, Prüfung Buchvorschläge über SQS

Tabelle 1 – Übersicht der Aufgaben und Technologien

Dabei haben alle Services unabhängige Datenbanken und reden über insgesamt vier Queues miteinander. Hintergrund dieser Umsetzung war es, die einzelnen Services möglichst praxisnah umzusetzen und sie somit so modular und unabhängig wie möglich zu gestalten. Jeder Service hat seine eigene Datenbank und greift nur auf diese zu. Braucht er Daten eines anderen Services, bekommt er eine Kopie über eine FIFO-Queue bei Amazon SQS. In Abbildung 5 ist die Kommunikation nochmal übersichtlich dargestellt:

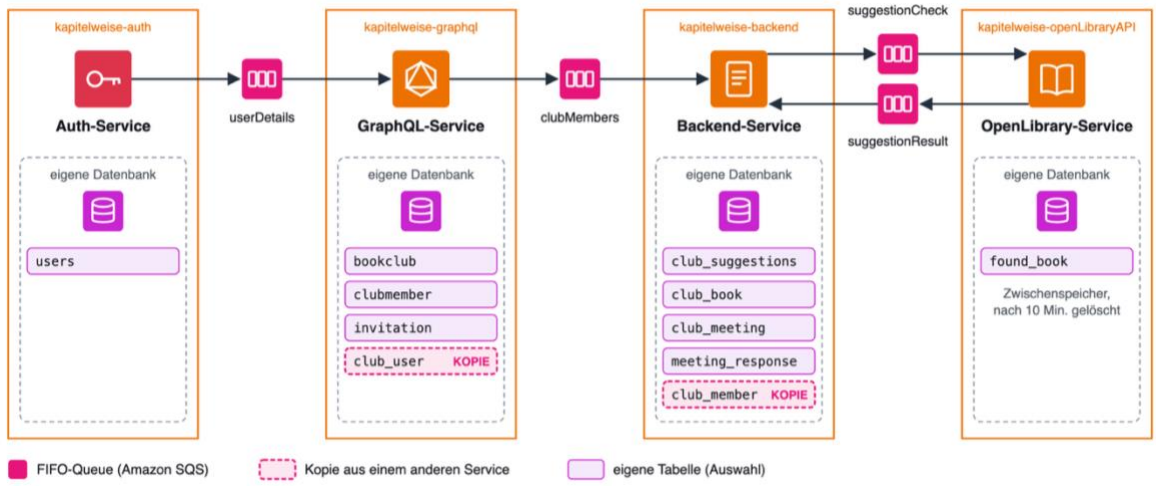


Abbildung 5 – Architekturdiagramm Kapitelweise – Darstellung der Queues. (KI-generiert).

Welche Daten wir dort austauschen und warum uns unser Lernprozess dort hingebracht hat, wird in den späteren Kapiteln erläutert.

3 API-Design

- Kapitelweise besteht aus vier Services mit eigener Basis-URL. Ein gemeinsames Präfix wie /api gibt es nicht. Alle tauschen Daten als JSON aus.
- Authentifizierung per Token: Alle Endpunkte außer POST /register, POST /login und /alexa erfordern den Header Authorization: Bearer <token>. Den Token stellt der Auth-Service aus, die anderen Services prüfen ihn selbst.
- Der Nutzer wird immer aus dem Token bestimmt. Ein Token oder die eigene userId stehen nie in den Request-Feldern.
- Parameter aus dem Pfad (z. B. clubId) werden nicht zusätzlich im Body geschickt. GET-Anfragen haben keinen Body.
- GraphQL antwortet auch bei Fehlern mit 200. Der Fehler steht dann im Feld errors, die Art in extensions.classification.

Statuscodes

200	Anfrage erfolgreich, Daten in der Antwort
201	Termin angelegt
400	Ungültige Eingaben
401	Kein, ungültiger oder abgelaufener Token bzw. falsche Zugangsdaten
403	Keine Berechtigung (z. B. kein Admin oder kein Mitglied des Clubs)
404	Ressource nicht gefunden
409	Konflikt mit dem aktuellen Stand (z. B. Name vergeben, schon ein aktuelles Buch)
504	Externe Buch-API nicht erreichbar

Tabelle 2 – Übersicht Statuscodes

Auth-Service

Endpunkt: https://kapitelweise-auth-7bf808281135.herokuapp.com/

Method	URI	Description	Data In	Data Out
POST	/register	Registrieren	username, password	200 - id, username, created_at Fehler: 400, 409
POST	/login	Anmelden	username, password	200 - token (10 Stunden gültig) Fehler: 401

Tabelle 3 – API Auth-Service

GraphQL-Service

Endpunkt: https://kapitelweise-graphql-f85172125d1d.herokuapp.com/graphql

Alle Operationen laufen als POST an /graphql. Die Spalte Method nennt die Art der Operation, URI ihren Namen. Fehler stehen im Feld errors, die Art in extensions.classification.

Method	URI	Description	Data In	Data Out
Query	member	Startseite laden: eigener Name, Clubs und offene Einladungen	-	id, name, membership (id, name, description), invitationPending (id, createdAt, bookclub, invitedBy) Fehler: NOT_FOUND
Query	bookclub(id)	Details eines Clubs	id	id, bookClubName, description, role (ADMIN oder MEMBER) Fehler: FORBIDDEN
Mutation	createNewBookClub	Buchclub gründen, der Gründer wird Admin	bookClubInput: name, description	id, name, description Fehler: BAD_REQUEST
Mutation	responseInvitation	Einladung annehmen oder ablehnen	invitationInput: invitationId, status (ACCEPTED, DECLINED)	Club (id, name, description) bei ACCEPTED, null bei DECLINED Fehler: NOT_FOUND
Mutation	inviteUser	Nutzer in einen Club einladen (Admin)	inviteUserInput: bookclubId, username	id, invitationStatus, invitedUser, createdAt Fehler: BAD_REQUEST, FORBIDDEN, NOT_FOUND, INTERNAL_ERROR
Mutation	leaveBookClub	Eigenen Buchclub verlassen (nur für Mitglieder, nicht für Admin)	clubId: ID!	Boolean: true bei Erfolg Fehler: Club nicht gefunden, Nutzer kein Mitglied oder Nutzer ist Admin

Tabelle 4 – API GraphQL

Backend-Service

Endpunkt: https://kapitelweise-update-2dcfa83e8351.herokuapp.com/

Method	URI	Description	Data In	Data Out
POST	/clubs/{clubId}/proposals	Buch vorschlagen, ersetzt den eigenen alten Vorschlag	title, isbn, author, pages, firstPublishYear	200 - id, suggestion_status (PENDING), title, isbn, pages, author, first_publish_year, suggested_by Fehler: 400, 403, 409
GET	/clubs/{clubId}/proposals	Geprüfte Vorschläge und den eigenen Vorschlag anzeigen	-	200 - Liste mit id, suggestion_status, title, isbn, pages, author, first_publish_year, suggested_by Fehler: 403
POST	/clubs/{clubId}/current-book/{proposalId}	Vorschlag als aktuelles Buch auswählen (Admin)	-	200 - id, book_status, title, isbn, pages, author, first_publish_year, started_at, finished_at Fehler: 403, 404, 409
GET	/clubs/{clubId}/current-book	Aktuelles Buch anzeigen	-	200 - id, book_status, title, isbn, pages, author, first_publish_year, started_at, finished_at Fehler: 403, 409 (kein aktuelles Buch)
PATCH	/clubs/{clubId}/close-book	Aktuelles Buch abschließen (Admin)	-	200 - Book Closed Fehler: 403, 409
POST	/clubs/{clubId}/meetings	Termin anlegen (Admin)	title, description, starts_at, ends_at, location	201 - id, book_id, title, description, starts_at, ends_at, location Fehler: 400, 403
GET	/clubs/{clubId}/meetings	Kommende Termine anzeigen	-	200 - Liste mit id, book_id, title, description, starts_at, ends_at, location, my_attendance_status Fehler: 403
PUT	/clubs/{clubId}/meetings/{meetingId}/attendance	Zu- oder absagen	status (ACCEPTED, DECLINED)	200 - status Fehler: 400, 403, 404, 409 (Termin hat begonnen)

Method	URI	Description	Data In	Data Out
GET	/clubs/{clubId}/meetings/{meetingId}/attendance	Zu- und Absagen eines Termins anzeigen (Admin)	-	200 - Liste mit meetingId, username, status, respondedAt Fehler: 403, 404
GET	/alexa	Erreichbarkeit für Alexa prüfen (ohne Token)	-	200 - Alexa-Endpunkt erreichbar
POST	/alexa	Anfragen des Alexa-Skills (ohne Token)	Alexa-Request (Intent, Slots)	200 - Alexa-Response mit dem vorzulesenden Text

Tabelle 5 – API Backend-Service

OpenLibrary-Service

Endpunkt: https://kapitelweise-bookapi-38e6cb97d9cf.herokuapp.com/

Method	URI	Description	Data In	Data Out
GET	/book-search	Bücher über die Open Library suchen (max. 10 Treffer)	Query-Parameter: title, author, isbn, q	200 - docs: Liste mit foundBookId, title, author, isbn, pages, firstPublishYear, coverId Fehler: 400, 504

Tabelle 6 – API OpenLibrary-Service

Die alten Rest-Endpunkte, die am 19.10. durch GraphQL ersetzt wurden, sind aus Übersichtlichkeitsgründen nicht mehr in diesem Dokument, können bei Bedarf aber nachgereicht werden.

4 Implementierung - unser Lern- und Projektverlauf

Die Vorstellung erfolgt gegliedert nach Komponenten bzw. Services. Der zeitliche Verlauf ist dennoch in Tabelle 7 dargestellt und zeigt das agile Vorgehen.

Zeitraum	Phase
Sprint 1 - KW 40	Erstellung ERM-Modell, Planung und Feinschliff der Features. Erstellen gemeinsamer Repositories, Login und Registrierung, Aufbau Homepage
Sprint 2 - KW 41	Buchclubgründung, Einladungsverwaltung, Auth als eigenen Service
Sprint 3 - KW 42	Schnittstelle zu Open-Library, Einbindung GraphQL für Homepage
Sprint 4 - KW 43	Homepage zur SPA umbauen, Suche, Auswahl und Abschluss von Büchern
Sprint 5 - KW 44	Einbau Alexa-API, Termine erstellen und verwalten, Service-Worker und IndexedDB für Offline-Nutzung.
Sprint 6 - KW 45	Feinschliff, Tests, Service-Worker für MPA und Offline-Meldung.

Tabelle 7 – Übersicht der Sprintwochen

Wir wollten den Großteil des Codes selbst schreiben und nicht nur aus Ihren Vorlagen kopieren. Die nötigen Informationen haben wir uns aus dem Internet, aus YouTube-Videos (Liste im Quellenverzeichnis) und mit Hilfe von KI zusammengesucht. KI haben wir vor allem für die Ideenfindung, beim Design und beim Debugging eingesetzt, um möglichst nah an praxisnahem Best Practice umzusetzen.

Übergreifende Architektur. Die Orientierung an den MACH-Prinzipien wurde im Laufe des Projektes immer wichtiger für uns. Anfangs war das Backend weitgehend monolithisch aufgebaut. Unter anderem durch die Einführung von GraphQL haben wir es nachträglich in vier Services aufgeteilt: Auth-, Backend-, GraphQL- und OpenLibrary-Service.

Zunächst haben diese Services allerdings noch über eine gemeinsame Datenbank zusammengearbeitet. Der Auth-Service und der Club-Service haben beispielsweise auf dieselbe User-Tabelle zugegriffen und mussten deshalb keine Daten austauschen.

Schichtenarchitektur in jedem Service. Zum Projektstart hatte keiner von uns Erfahrung mit einem Java-Backend in Spring Boot. Aus unserem Einführungsmaterial[1] stammen der Aufbau über start.spring.io und die Schichtenstruktur aller Services (vgl. Abbildung 6) Controller nehmen Anfragen entgegen, Services enthalten Logik und Validierung, DAOs kapseln das SQL mit JdbcTemplate, und getrennte Domain-, Request- und Response-Klassen bilden Datenbankobjekt, Input und Output ab. Die Trennung kam erst, als wir mit der reinen DAO-Klasse auf Probleme stießen, durch Einarbeitung und KI, angelehnt an die Model-Klassen der Vorlage. Das sind mehr Klassen, aber jede hat eine eindeutige Aufgabe. Lokal testeten wir mit

[1] Devtiro 2023, o. S.

PostgreSQL und schema.sql: schnell, aber Probleme der Zielumgebung zeigten sich erst beim Deployment (4.1 und 4.2).

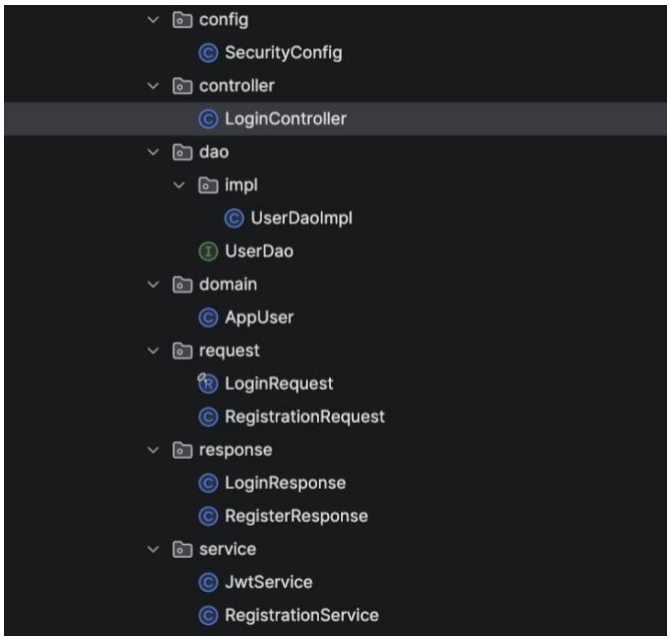


Abbildung 6 – Beispiel grundlegender Aufbau

War die Aufteilung richtig? Wir würden die Trennung wieder so machen, weil sich jeder Service ändern lässt, ohne die anderen zu beeinflussen, solange die Nachrichtenformate der Queues gleichbleiben. Der Preis sind mehr Repositories, vier Datenbanken, vier Queues und Kopien von Daten. Anders würden wir vor dem ersten Code festlegen, welcher Service welche Daten besitzt, die Fehlerbehandlung der Queues (Dead-Letter-Queue) von Anfang an mitplanen und die Zielumgebung früh einbeziehen.

4.1 Kapitelweise-Auth

Login und Registrierung setzen wir mit Spring Security und einem JSON Web Token (JWT) um.[2] Zunächst lief der Login per HTTP-Basic Auth über Postman: Dabei wird bei jeder Anfrage das Passwort mitgeschickt, und jeder Service müsste es selbst prüfen. Das JWT ist ein signierten Token mit zehn Stunden Gültigkeit (in einer echten Anwendung deutlich kürzer), den jeder Service selbst prüfen kann (vgl. Abbildung 7).

[2] Telusko 2024, o.S.

```java
@Service
public class JwtService {

    private static final String SECRET_KEY = "mysecretkeymysecretkeymyxwsecretkeymysecretkey";
    // muss so lange sein, sonst ist es bei mir auf Fehler gelaufen, später vllt. einen neuen erzeugen lassen
    //bzw. eigentlich Umgebungsvariable, aber für Testzwecke erstmal so

public String generateToken(String username){
    return Jwts.builder()
        .setSubject(username)
        .setIssuedAt(new java.util.Date())
        .setExpiration(new java.util.Date(System.currentTimeMillis() + 1000 * 60 * 60 * 10))
        .addClaims(new java.util.HashMap<>())
        .signWith(getSignedKey(), SignatureAlgorithm.HS256)
        .compact();
```

Abbildung 7 – JWT-Service

Entscheidend dabei war für uns das Verständnis der Filterkette (vgl. Abbildung 8). Durch Freigaben einzelner Routen konnten wir unsere neuen Features immer auch ohne Token testen. Außerdem nutzen wir die Funktion des JwtFilters da er den Nutzer immer in den Kontext einträgt, wenn der Token gültig ist. Im Controller holen wir den Nutzernamen deshalb immer mit @AuthenticationPrincipal String username.

```java
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
            .cors(Customizer.withDefaults())
            .csrf(csrf -> csrf.disable())
            .logout(logout -> logout.disable())
            .authorizeHttpRequests(auth -> auth
                    .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()
                    // der Auth-Service hat nur öffentliche Endpunkte, alles andere wird gesperrt
                    .requestMatchers("/login", "/register", "/error").permitAll()
                    .anyRequest().denyAll())
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .exceptionHandling(e -> e.authenticationEntryPoint(
    new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)));
```

Abbildung 8 – Spring Secure Filter-Chain

Aufteilung in einen eigenen Service

Anfangs lag der Auth-Service im selben Backend-Repository wie die übrigen Komponenten, das wollten wir aufgrund von MACH ändern.

Das JWT stellt seitdem ausschließlich der Auth-Service aus, alle anderen Services prüfen den Token nur. Dafür haben wir SecurityConfig, JwtFilter und JwtService in jeden Service kopiert, alle verwenden denselben geheimen Schlüssel.

Da nur der Auth-Service auf die User-Tabelle zugreifen kann, der Club-Service userId und username aber ebenfalls benötigt, schreibt der Auth-Service nach dem Anlegen eines Users eine Nachricht in die FIFO-Queue userDetails.fifo. Für FIFO haben wir uns entschieden, weil

eine Standard-Queue die Reihenfolge nicht garantiert. Bei einer späteren Erweiterung, etwa dem Löschen eines Users, könnte sonst das Löschen vor dem Anlegen ankommen.

Das Cookie-Problem. Wir wollten das JWT als Cookie ausliefern (vgl. Abbildung 9). Der Browser hängt es automatisch an jede Anfrage an, und mit HttpOnly kann JavaScript es nicht lesen.

```
@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody LoginRequest request,
                               @Value("${cookie.secure:false}") boolean cookieSecure)
    authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(request.username(), request.password()
    String token = jwtService.generateToken(request.username());

    ResponseCookie cookie = ResponseCookie.from("token", token)
            .httpOnly(true)
            .secure(cookieSecure)
            .sameSite("Lax")
            .path("/")
            .maxAge(Duration.ofHours(10))
            .build();
```

Abbildung 9 – Cookie Einbindung

Lokal funktionierte das, auf Heroku nicht: Der Login war erfolgreich (HTTP 200), die Folgeanfragen lieferten aber 401. Die Ursache haben wir erst später mit Hilfe von KI verstanden (vgl. Abbildung 10).



Abbildung 10 – Probleme mit dem Cookie

Frontend und Backend laufen unter zwei Subdomains von herokuapp.com. Dabei haben wir auch den Unterschied zwischen Site und Origin gelernt. Lokal liefen beide Teile unter demselben Rechnernamen, deshalb fiel es erst auf Heroku auf. Eine eigene Domain oder ein Reverse-Proxy waren uns zu aufwendig, daher haben wir am 11.10. zurückgesetzt: Der Token kommt im Response-Body zurück und wird als Authorization: Bearer gesendet.

War die Cookie-Entscheidung falsch? Die Grundidee war für uns trotzdem spannend und daher war es zwar mehr Aufwand, aber eine spannende Erkenntnis. Falsch war, dass wir uns vorher nicht tiefergehend informiert haben, wie Cookies zwischen verschiedenen Domains funktionieren. Vor allem aber hätten wir den Login direkt zu Beginn auf Heroku deployen sollen.

4.2 Kapitelweise-Backend

Das Kapitelweise-Backend war unser erster Service. Wie der Name schon impliziert, hatten wir zu diesem Zeitpunkt noch keine Planung bezüglich mehrerer Services mit eigenen Datenbanken. Am Ende stellt es die Buchclub-Detailseite dar, der Buchvorschläge entgegennimmt, das Buch abschließt und das neue auswählt.

Nutzung der Config-Files. In den ersten beiden Sprints wurde hauptsächlich in diesem Backend gearbeitet. Dies hatte zur Folge, dass wir immer wieder lokal testen wollten, aber die postgres-Datenbank jeweils anders konfiguriert hatten. Anfangs haben wir dabei jedes Mal die application.yml Datei manuell umgeschrieben. Dies haben wir dann zeitnah durch unterschiedliche application.yml und durch die Profileinstellung vereinfacht. Zusätzlich haben wir das Konzept der Umgebungsvariablen kennengelernt. Ehrlicherweise haben wir das lange ignoriert, im Zuge des Heroku-Deployments aber umgestellt. Im nächsten Projekt würden wir Profile und Umgebungsvariablen am ersten Tag anlegen (vgl. Abbildung 11).



Abbildung 11 - Erstellung in der application.yml

Aufteilung der Services und Synchronisierung der Mitgliederdaten

Hier lagen zuerst Login, Registrierung, Buchclub-Gründung und Einladungen. Im Zuge von MACH haben wir den Auth-Service ausgelagert und alle Abfragen der Startseite auf GraphQL umgestellt und in ein eigenes Repository verschoben. Dabei haben wir zunächst strikt nach Seiten geschnitten. Beide Services brauchten aber fast dieselben Daten, eine eigene Datenbank pro Service hätte so keinen Sinn gemacht. Deshalb haben wir nach Daten getrennt: Clubs, Mitglieder und Einladungen gehören dem GraphQL-Service, Vorschläge und Bücher dem Backend-Service. Das Backend muss aber prüfen, ob jemand Mitglied oder Admin ist, das lösen wir über eine Kopie der Mitgliederdaten, die über die Queue „clubMembers.fifo“ aktualisiert wird. Damit das Backend zwischen einem Beitritt und einem Austritt unterscheiden kann, enthält jede Nachricht einen der Typen: „MEMBER_JOINED“ oder „MEMBER_LEFT“. Dass wir die Mitgliederdaten dafür doppelt speichern müssen, fanden wir zuerst nicht ideal. Bei getrennten Services ist das nach unserer Recherche aber ein gängiger Weg.[3]

[3] Microsoft o.J., o.S.

Buchvorschläge über die Queue. Weil der Buchvorschlag über zwei Services läuft, haben wir den Ablauf vorher ausführlich geplant. Die Daten aus dem Frontend sollen nicht manipuliert sein, deshalb prüft der OpenLibrary-Service jeden Vorschlag. Synchron dauerte das 5 bis 10 Sekunden, so lange sollte der User nicht warten. Deshalb läuft die Prüfung jetzt asynchron:

1. Der Backend-Service speichert mit Status PENDING und antwortet sofort.

```
    SuggestionsResponse response;
    try {
        response = bookclubDao.saveBooksSuggestion(clubId, request, username);
    } catch (DuplicateKeyException e) {
        log.warn("Vorschlag von " + username + " in Club " + clubId + " wird schon gespeichert");
        throw new ResponseStatusException(HttpStatus.CONFLICT, "Dein Vorschlag wird gerade schon gespeichert!");
    }
    log.info("Vorschlag " + response.getId() + " von " + username + " in Club " + clubId + " gespeichert");

    amazonSQSService.sendSuggestionCheck("SEND_SUGGESTION", response.getId(),clubId, response.getIsbn());

    return response;
}
```

Abbildung 12 – Vorschlagsverarbeitung – Direkte Antwort

2. Er schickt ein SuggestionEvent an die Queue suggestionCheck.fifo.

```
public void sendSuggestionCheck(String type, Long suggestionId, Long bookClubId, String isbn){

    try {
        String body = jsonMapper.writeValueAsString(new SuggestionEvent(type, suggestionId, bookClubId, isbn));
        SendMessageRequest send = new SendMessageRequest()
                .withQueueUrl(SUGGESTION_CHECK_URL)
                .withMessageBody(body)
                .withMessageGroupId(String.valueOf(suggestionId))
                .withMessageDeduplicationId(String.valueOf(suggestionId));
        sqs.sendMessage(send);
```

Abbildung 13 - Vorschlagsverarbeitung – Event an die Queue

3. Der OpenLibrary-Service prüft die ISBN und antwortet über suggestionResult.fifo.
4. Das Backend setzt den Vorschlag auf READY/REJECTED und übernimmt die geprüften Daten,

```
    List<Message> messages = sqs.receiveMessage(request).getMessages();
    for (Message message : messages) {
        log.info("SQS-Nachricht von SuggestionResultQueue empfangen: " + message.getBody());
        try {
            SuggestionResultEvent event = jsonMapper.readValue(message.getBody(), SuggestionResultEvent.class);

            if (event.type().equals("SUGGESTION_RESULT")) {
                if (event.status().equals("READY")) {
                    bookclubDao.markSuggestionReady(event.suggestionId(), event.title(), event.author(), event.pages(),
                } else {
                    bookclubDao.markSuggestionRejected(event.suggestionId());
```

Abbildung 14 - Vorschlagsverarbeitung – Verarbeitung der Antwort

Es war die komplizierteste Umsetzung, aber auch die am meisten durchgeplante, und das hat sehr geholfen. Aus unserer Sicht ist es unsere stärkste architektonische Entscheidung.

Zum zweiten Mal zu spät deployt. Lokal hatten wir die Daten aller Services in einer Datenbank, aber in getrennten Schemas, jeweils mit einer eigenen Datenbankrolle. Beim Deployment am 25.10. stellte sich heraus, dass das auf Heroku nicht geht, weil wir dort keine eigenen Rollen anlegen können. Wir haben die Schemas wieder entfernt, jetzt hat jeder Service seine eigene Datenbank.

Integration Alexa. Wir entschieden uns bei der Integration eines weiteren externen Services für die Integration einer Amazon Alexa Schnittstelle. Hierfür wurden die Intents ClubsIntent zum Ausgeben der aktuellen Buchclubs, CurrentBookIntent zum Ausgeben des aktuellen Buchs eines Buchclubs und SuggestionIntent zum Ausgeben der Vorschläge eines Members und eines Buchclubs implementiert. Die Struktur wurde dabei von dem Demo Repository übernommen und angepasst.

Im Rahmen mehrerer Tests stellte sich heraus, dass der reguläre Alexa Nutzungsfluss die Anmeldung eines Users aufgrund fehlender Authentifizierungsmechanismen nicht unterstützt. Für die Abfragen der Vorschläge und Bücher wird ein JWT erwartet. Amazon Alexa erhält diesen Token über ein „OAuth-2-Verfahren“. Die erforderlichen Authentifizierungsmechanismen sind in der Anwendung bislang nicht implementiert. Aus Kapazitäts- bzw. Zeitgründen wird dies auch nicht mehr umgesetzt. Um die Intents ohne Account Linking zu testen, wird der JWT manuell über JSON übergeben.

Was wir daraus mitnehmen:

Genauso machen würden wir es bei komplexen Abläufen: Wir zeichnen sie vor dem Programmieren gemeinsam auf, wie beim Buchvorschlag, das hat sich bewährt. **Anders machen würden wir** es bei tiefgehenden Änderungen. Hier planen wir Heroku als Zielumgebung von Anfang an mit und deployen häufiger. Umgebungsvariablen nutzen wir gleich zu Beginn, statt sie erst später nachzuziehen. Den Code würden wir regelmäßig reviewen, sonst hätten wir die hängenden Vorschläge nicht gefunden. Und die Integration externer Services gehen wir früher an, damit wir ihre Voraussetzungen rechtzeitig einplanen können.

4.3 Kapitelweise-GraphQL

Der GraphQL-Service liefert, was die Startseite braucht: den Nutzer, seine Buchclubs und seine offenen Einladungen. Außerdem können Nutzer hier Clubs gründen und Einladungen verschicken oder beantworten. Vorher waren dafür vier Abfragen an verschiedene Endpunkte nötig (Underfetching). Aus der Vorlesung hatten wir mitgenommen, dass GraphQL das löst: Das Frontend fordert in einer Abfrage genau die Felder an, die es braucht. Außerdem bereitete es die SPA vor. Mit GraphQL hatte vorher keiner von uns gearbeitet.

Vom Endpunkt zum Graphen. Anfangs haben wir GraphQL wie REST behandelt, mit einer eigenen Query pro Feld. Mit Hilfe einer Online-Einführung haben wir es dann besser verstanden.[4] Es gibt einen Einstieg member, und die Felder darunter lädt GraphQL über @SchemaMapping nur, wenn das Frontend sie abfragt (vgl. Abbildung 16). Das Schema zwang uns außerdem zu überlegen, was die Startseite wirklich braucht, etwa keine Rolle und keinen Einladungsstatus. Die DAOs aus dem Backend konnten wir fast unverändert übernehmen.

```
type Query{
    member: Member!
    bookclub(id: ID!): BookclubDetail!
}
```

```
type Member{
    id: ID!
    name: String!
    membership: [Bookclub!]!
    invitationPending: [Invitation!]!
}
```

Abbildung 15 – GraphQL Implementierung

```
@QueryMapping(name="member")
public Member findName(@AuthenticationPrincipal String username) {
    return userService.returnMember(username);
}


@SchemaMapping(typeName = "Member", field="membership")
public List<BookClubResponse> bookclub(@AuthenticationPrincipal String username){
    return bookClubService.getClubs(username);
}
```

Abbildung 16 – GraphQL SchemaMapping

Fehler funktionieren anders. Beim ersten Test kam nur INTERNAL_ERROR, denn GraphQL antwortet fast immer mit HTTP 200 und schreibt Fehler in das Feld errors. Ein DataFetcherExceptionResolver übersetzt jetzt unsere Exceptions. Bei einem Review fiel auf, dass ein 409 weiterhin als INTERNAL_ERROR erscheint. Außerdem fehlte ein Mitglied im Backend für immer, wenn das Senden in die Queue scheiterte. Diese Lücke hatten wir in 3.3.3 schon geschlossen, die Lösung aber nicht übertragen. Jetzt sendet auch hier ein Scheduler hängende Mitglieder erneut.

Was wir daraus mitnehmen:

Genauso würden wir die Planung der Umsetzung machen. Wir haben hier lange überlegt, wie wir die Daten aufbauen und was wir genau brauchen. Das würde wir für jedes API-Design in Zukunft wieder so machen. **Anders** würden wir uns vor dem ersten Code anschauen, wie die neue Technik mit Fehlern umgeht. Zusätzlich würden wir bei gefundenen Fehlern in einem Service prüfen, ob wir diese Fehler auch in einem anderen Service haben.

[4] Dan Vega 2026, o.S.

4.4 Kapitelweise-OpenLibrary

Der OpenLibrary-Service verbindet unsere Anwendung mit der externen API der Open Library. Es hat dabei grundsätzlich zwei Aufgaben, es sucht Bücher über die externe API und prüft im Hintergrund jeden Buchvorschlag, bevor dieser im Backend als READY gilt.

Fremde Daten verstehen. Wir hatten eine einfache Liste von Büchern erwartet. Tatsächlich heißt die Liste docs, ein Treffer ist ein „Werk“ und keine konkrete Ausgabe, und eine ISBN gibt es nur bei den Ausgaben (editions). Weil wir Vorschläge über die ISBN prüfen wollten, haben wir nachträglich eigene Klassen für die Ausgaben gebaut und suchen aus jedem Werk die erste Ausgabe mit ISBN heraus (vgl. Abbildung 17 und Abbildung 18, vorher und nachher). Treffer ohne ISBN oder ohne deutsche Ausgabe filtern wir heraus. Wir hätten die Antwort vorher in Postman genauer prüfen sollen.

Vorher:


```
"docs": [
    {
        "author_key": [
            "OL4926626A",
            "OL2838892A",
            "OL6812841A"
        ],
        "author_name": [
            "Gregory Bassham",
            "Eric Bronson",
            "William Irwin"
        ],
        "ebook_access": "no_ebook",
        "edition_count": 2,
        "first_publish_year": 2012,
        "has_fulltext": false,
        "key": "/works/OL21509535W",
        "language": [
            "ger"
        ],
        "public_scan_b": false,
        "title": "Die Philosophie Bei der Hobbit"
    }
```


Abbildung 17 – OpenLibrary Antwort vorher

Nachher:


```
{
    "docs": [
        {
            "foundBookId": 184,
            "title": "Die Philosophie Bei der Hobbit",
            "author": "Gregory Bassham, Eric Bronson, William Irwin",
            "isbn": "9783527672295",
            "pages": 276,
            "firstPublishYear": 2012
        }
    ]
}
```


Abbildung 18 – OpenLibrary Antwort nachher

Wem gehören die gefundenen Bücher? Jedes Buch, das die Suche ans Frontend gibt, speichern wir in found_book. So kann der Service prüfen, ob ein Vorschlag wirklich aus einer

Suche stammt, statt den Daten aus dem Frontend zu vertrauen. Zuerst lag die Tabelle im Backend-Service. Wir haben sie am selben Tag wieder entfernt, weil die Daten zu dem Service gehören, der sie erzeugt. Ein Scheduler löscht die Einträge nach 10 Minuten, fehlt ein Buch dann, fragen wir die Open-Library-API erneut ab (vgl. Abbildung 19).

```
if(found.isEmpty()){
    BookSearchResponse result = bookSearchService.searchBooks(new BookSearchRequest(null, null, null, event.isbn()));

    if (result != null && result.getDocs() != null && !result.getDocs().isEmpty()) {
        BookResponse book = result.getDocs().get(0);
        found = Optional.of(new FoundBook(book.getTitle(), book.getAuthor(), book.getIsbn(), book.getPages(), book.getFirstPublishYear()));
    }
}
```

Abbildung 19 – OpenLibrary erneute Abfrage

Zwei Arbeitsweisen in einem Repository. Den Service hat zuerst ein Teammitglied gebaut, die Queue-Anbindung kam später von einem anderen. Beim Zusammenführen unterschieden sich unsere Arbeitsweisen (kleingeschriebene Klassen, Fehler im Jackson-Mapper), das hat einen Abend gekostet.

Was wir daraus mitnehmen

Genauso würden wir Daten aus dem Frontend nicht ungeprüft übernehmen, sondern gegen eigene Daten prüfen. Der Zwischenspeicher in found_book hat sich dafür bewährt. **Anders** würden wir die Anforderungen und die echte Antwort einer fremden API vor dem ersten Code genau lesen und am Projektstart gemeinsame Konventionen für alle Repositories festlegen.

4.5 Kapitelweise-Frontend

Unser Frontend ist eine Mischform: Login, Registrierung und die Buchclub-Detailseite sind klassische Einzelseiten aus HTML, Bootstrap und JavaScript (MPA), nur die Startseite ist eine Angular-SPA, die ihre Daten über GraphQL holt. Das hat sich angeboten, weil GraphQL die Daten gebündelt liefert. Die Detailseite umzubauen hätte die Zeit nicht zugelassen. Rückblickend war das vertretbar, hat aber den Offline-Modus erschwert. Die SPA läuft unter /public/spa/ neben den alten Seiten, dafür mussten wir den Angular-Build dorthin kopieren und den base href anpassen. Wir nutzen Routing mit # in der URL, damit sie auf Heroku funktioniert, ohne dass der Server die Angular-Routen kennt. Beim Öffnen flimmerte zuerst kurz die Startseite vor der Weiterleitung zum Login, erst eine Prüfung des Tokens vor dem Aufbau der Oberfläche hat das behoben.

Für die Fallstudie haben wir einen Offline-Modus mit zwei Service-Workern und IndexedDB ergänzt. Das war lehrreich, denn beide Begriffe kannten wir vorher nicht, die Aufteilung in MPA und SPA hat es aber erschwert. **Was wir daraus mitnehmen.** Die Aufteilung mit GraphQL und SPA würden wir wieder wählen. In einem eigenen Projekt würden wir uns entweder für MPA oder SPA entscheiden, um keine Technologien zu vermischen.

5 Wissenschaftliche Reflexion der Teammitglieder

Niklas Ültzhöfer / 8485275

Habe ich strukturiert und organisiert gearbeitet?

Aus meiner Sicht und dem, was ich sonst von mir gewohnt bin, habe ich deutlich strukturierter gearbeitet. Ich programmiere auch in meiner Freizeit und verzichte dort in der Regel auf GitHub Issues, API-Dokumentation und größere Architektur-Planung. Das führte häufig dazu, dass ich bereits geschrieben Code großflächig umbauen musste, um alles sauber umsetzen zu können.

Innerhalb des Projekts habe ich gemerkt, dass die Planung über Tickets, die klare Verteilung der Aufgaben in einem Sprint und die API-Doku ein großer Effizienz-Booster sind. Natürlich lief auch dieses Mal nicht alles perfekt, aber mir hat es sehr dabei geholfen die einzelnen Schritte besser nachzuvollziehen und strukturierter zu programmieren.

Den größten Hebel dabei hatte für mich die API-Dokumentation. Dadurch konnten viele aufkommende Fragen während der Implementierung im Backend sofort geklärt werden und der Abstimmungsbedarf mit dem Frontend war minimal. Zusätzlich hatte man auch noch 1-2 Wochen nach der Implementierung eine saubere Übersicht. Auch die Postman-Tests liefen dadurch deutlich zielorientierter.

Weniger gut gelungen ist mir die Konzentration auf eine Sache. Es passiert mir dabei öfter, dass ich eigentlich an einem Ticket arbeite, mir dann aber etwas anderes auffällt. Häufig möchte ich das dann direkt umsetzen und verliere die Konzentration für das eigentliche Ticket. Das ist mir auch in diesem Projekt wieder passiert. In diesem Bereich muss ich deutlich strukturierter werden und gefundene Themen einfach notieren und zurückstellen. Dennoch haben die Github-Issues dabei zumindest manchmal geholfen, da man dort gefundene Themen sofort eintragen konnte.

Wo liegen meine Stärken und Schwächen?

Stärken. Meine Stärke in diesem Projekt lag aus meiner Sicht in der Rolle des Projektleiters. Ich habe die wöchentlichen Tickets vorbereitet, deren Umsetzung im Auge behalten und dabei unser Endziel immer im Blick behalten. Fachlich habe ich mich außerdem in Java sicher gefühlt und habe viele der komplexeren Themen umgesetzt.

Schwächen. Ich bin häufig ungeduldig und arbeite manchmal „schnell schnell“. Das ist ähnlich zu dem, was ich anhand der Struktur schon erwähnt hatte. Bei Fehlern frage ich zusätzlich manchmal zu schnell die KI, statt zuerst selbst zu debuggen. Das kostet Lernerfolg, weil ich die Ursache eines Fehlers dann weniger selbst nachvollziehe. In Zukunft möchte ich wieder zuerst selbst debuggen und das Problem tiefgreifend verstehen (bestenfalls lösen), bevor ich die KI frage.

Was würde ich in einem ähnlichen Projekt anders machen?

Ich würde die Architektur deutlich detaillierter planen. Insbesondere der GraphQL-Umbau war nicht vollständig durchdacht und geplant und hat uns uns damit ein bis zwei Wochen beschäftigt und viele Folgefehler verursacht. Dadurch waren die Sprintwochen 3 und 4 sehr aufwändig und durcheinander.

Außerdem habe ich zu Beginn den Umfang überschätzt. Ich würde weniger Features ins Projekt nehmen und diese dafür genauer planen. Zusätzlich würde ich schon am Anfang ein kleines Architekturdiagramm erstellen. So hätten alle dasselbe Bild vom System, und eine große Änderung wie der GraphQL-Umbau ließe sich vorab auf ihre Folgen prüfen.

Was würde ich Studierenden des nächsten Jahres empfehlen?

- So früh wie möglich anfangen. Nur so bleibt Zeit für Planung und für Änderungen, wie bei uns der GraphQL-Umbau.
- Sich gegenseitig helfen. Jede Person im Projekt sollte motiviert sein, den anderen zu helfen, sonst ist es nicht zu schaffen.
- Nicht nur die KI fragen. YouTube-Videos und andere Tutorials helfen, Zusammenhänge wirklich zu verstehen. Die KI ist eine gute Unterstützung, aber nur, wenn man versteht, was man übernimmt. Sonst bringt das Projekt keinen Lernerfolg.
- Aufgaben fair und sauber verteilen. Jede Person muss mitziehen.

Fazit

Das Projekt hat mir gezeigt, wie viel Struktur durch Tickets, API-Dokumentation und Sprints entsteht, und das meine Ungeduld die größte Gefahr für diese Struktur ist. Für künftige Projekte nehme ich mit: mehr Planung vor der Umsetzung, ein kleiner Architekturentwurf zu Beginn, weniger Features und beim Debuggen zuerst den eigenen Kopf benutzen. Meine koordinierende Rolle möchte ich beibehalten und dabei geduldiger und fokussierter arbeiten.

Fabrice Böhme / 1523396

Ich persönlich fand das Projekt spannend, und das hatte mehrere Gründe. Besonders gefallen hat mir, dass es nicht einfach nur darum ging, etwas zu programmieren. Dadurch, dass das Projekt mit der Fallstudie verbunden war, hat es sich vielmehr wie ein größeres praktisches Projekt angefühlt, wie man es auch in einem Unternehmen haben könnte. Man hatte verschiedene Anforderungen, musste sich Gedanken über die Umsetzung machen und gemeinsam Lösungen finden. Gerade diese Mischung aus Programmierung, Planung und Organisation fand ich sehr interessant.

Was mir besonders positiv in Erinnerung geblieben ist, war die Organisation innerhalb unserer Gruppe. Ich finde, dass wir von Anfang an ziemlich strukturiert gearbeitet haben. Durch die Verwendung von GitHub, die Erstellung von Tickets und Tasks sowie die Aufteilung in Frontend und Backend hatte jeder einen guten Überblick über seine Aufgaben. Auch die Arbeit in den einzelnen Sprints hat meiner Meinung nach sehr gut funktioniert. Die meisten Aufgaben konnten wir wie geplant erledigen, und wenn etwas doch länger gebraucht hat, konnten wir es relativ flexibel in den nächsten Sprint verschieben. Insgesamt hatte ich das Gefühl, dass wir als Gruppe wirklich produktiv zusammengearbeitet haben und jeder seinen Teil zum Projekt beitragen wollte. Auch fachlich konnte ich während des Projekts einiges dazulernen. Gerade die Arbeit mit verschiedenen Technologien und Frameworks hat mir gezeigt, wie hilfreich diese bei der Entwicklung sein können. Im Frontend war beispielsweise Bootstrap eine große Unterstützung, da man damit viele Dinge deutlich einfacher umsetzen konnte. Aber auch GitHub hat mir gezeigt, wie viele Möglichkeiten es eigentlich für die Zusammenarbeit an einem gemeinsamen Projekt gibt. Vorher war mir gar nicht bewusst, wie umfangreich man damit Aufgaben organisieren und die Entwicklung koordinieren kann.

Meine persönlichen Stärken sehe ich mittlerweile vor allem im Frontend. Durch die Arbeit mit HTML, CSS und JavaScript konnte ich mir ein gutes Verständnis dafür aufbauen, wie die einzelnen Bestandteile zusammenhängen und funktionieren. Eine Entscheidung, die mir dabei besonders geholfen hat, war die Überarbeitung unserer Ordnerstruktur. Am Anfang war der Code teilweise noch sehr unübersichtlich, da vieles in wenigen Dateien untergebracht war. Nachdem wir die einzelnen Bereiche aufgeteilt und beispielsweise API-Aufrufe und Seitenlogik voneinander getrennt hatten, wurde die Arbeit deutlich angenehmer. Rückblickend war das für mich eine der sinnvollsten Entscheidungen im Frontend.

Etwas schwerer habe ich mich dagegen im Backend getan. Als ich dort zum ersten Mal eigene Aufgaben übernehmen musste, war ich zunächst etwas überfordert, weil die Struktur und die

Arbeitsweise doch ziemlich anders waren als im Frontend. Dazu kam, dass ich das Frontend von Anfang an mit aufgebaut hatte und dadurch genau wusste, wie alles entstanden ist. Im Backend musste ich dagegen in bereits bestehenden Code einsteigen und erst einmal verstehen, wie die verschiedenen Klassen zusammenarbeiten. Im Nachhinein konnte ich meine Aufgaben trotzdem umsetzen und fand es auch interessant, mal einen Einblick in diesen Bereich zu bekommen.

Eine weitere große Herausforderung war für mich die ganze Thematik rund um SPA und Angular. Gerade am Anfang fiel es mir schwer, mich dort zurechtzufinden, da vieles neu war und die Zeit für die Umsetzung relativ begrenzt war. Mit Unterstützung und zusätzlichen Erklärungen konnte ich mich nach und nach besser einarbeiten, trotzdem würde ich das als eine meiner größten Herausforderungen während des Projekts bezeichnen.

Was ich außerdem mitgenommen habe, ist, dass manche Aufgaben deutlich mehr Zeit benötigen, als man anfangs erwartet. Zu Beginn hatten wir noch wesentlich mehr Funktionen geplant, die wir im Laufe des Projekts nicht alle umsetzen konnten. Trotzdem bin ich der Meinung, dass wir als Gruppe sehr produktiv gearbeitet haben. Besonders positiv fand ich, dass wir nicht nur unsere Aufgaben erledigen wollten, sondern auch wirklich Interesse an dem Projekt hatten und Spaß daran, es weiterzuentwickeln.

Wenn ich nochmal an einem ähnlichen Projekt arbeiten würde, würde ich versuchen, mich stärker im Backend einzubringen. Im Frontend fühle ich mich mittlerweile sicher, während ich im Backend noch einiges dazulernen kann. Außerdem würde ich von Anfang an etwas mehr Zeit für unerwartete Probleme einplanen, da sich während der Entwicklung immer wieder Dinge ergeben, mit denen man vorher nicht gerechnet hat.

Studenten aus dem nächsten Jahr würde ich auf jeden Fall empfehlen, sich ein Projektthema auszusuchen, an dem sie wirklich Interesse haben. Ich glaube, dass das einen großen Unterschied macht. Wenn man selbst Spaß an der Idee hat, ist man auch viel motivierter, sich mit Problemen auseinanderzusetzen und das Projekt weiterzubringen. Genauso wichtig finde ich eine gute Organisation im Team, weil dadurch vieles einfacher wird und man sich besser auf seine eigentlichen Aufgaben konzentrieren kann.

Insgesamt hat mir das Projekt sehr viel Spaß gemacht und ich konnte sowohl fachlich als auch bei der Zusammenarbeit im Team einiges mitnehmen. Vor allem fand ich es interessant, einmal den gesamten Ablauf eines größeren Projekts mitzuerleben und nicht nur einzelne Programmieraufgaben zu bearbeiten. Auch wenn nicht immer alles direkt funktioniert hat, war gerade das ein wichtiger Teil der Erfahrung.

Kira Kappes / 1768280

Während unseres Projekts „Buchclub“ habe ich hauptsächlich im Frontend mitgearbeitet und dort verschiedene Aufgaben übernommen. Zu Beginn war ich teilweise überfordert und wusste nicht genau, wie ich an die Aufgaben herangehen sollte. Ein Grund dafür war, dass ich zuvor noch nie an der Entwicklung eines Webservices mitgearbeitet hatte. Meine Vorkenntnisse beschränkten sich auf grundlegende Elemente in HTML und CSS.

Auch die Umsetzung einzelner Aufgaben fiel mir anfangs nicht immer leicht. Besonders schwierig war es, unterschiedliche Vorstellungen vom Aussehen der Anwendung miteinander abzustimmen bzw. aus den Aufgaben herauszulesen. Außerdem wollte ich die Aufgaben von Beginn an möglichst perfekt umsetzen und erst mit meinem Team kommunizieren, wenn die Aufgabe fertig ist. Dadurch habe ich teilweise zu viel Zeit in einzelne Aufgaben investiert, während das Team bereits mit den nächsten Schritten weitermachen wollte.

Im Laufe des Projekts habe ich gelernt, zunächst eine klare Grundstruktur aufzubauen und meinen Fortschritt regelmäßig mit den Teammitgliedern zu teilen – auch wenn er noch klein ist. So wissen alle, woran ich arbeite, und mögliche Missverständnisse können früh geklärt werden.

Außerdem habe ich erkannt, dass es wichtig ist, sich zunächst auf die Anforderungen im jeweiligen Ticket zu konzentrieren und keine zusätzlichen Funktionen einzuplanen, die nicht darin beschrieben sind. Eine Aufgabe kann schrittweise erweitert werden, wenn dafür Zeit und Bedarf bestehen.

Eine meiner größten Herausforderungen waren JavaScript und die Anbindung des Frontends an das Backend. Besonders die Arbeit mit den API-Schnittstellen fiel mir schwer, weil mir zu Beginn das nötige Wissen und Verständnis dafür fehlten. Ich war mir häufig nicht sicher, welche Schnittstelle ich für eine bestimmte Funktion benötigte und wie die einzelnen Teile des Frontends und Backends miteinander kommunizieren. Wenn ich bei der Umsetzung nicht weiterkam, bat ich teilweise eine KI um Unterstützung. Dabei wurde mir deutlich, dass ich die vorgeschlagenen Lösungen nachvollziehen und prüfen muss, damit ich sie sinnvoll einsetzen kann. Während ich bei der technischen Verbindung zwischen Frontend und Backend noch viel lernen musste, fiel mir die Gestaltung der Benutzeroberfläche leichter. Besonders die Umsetzung der HTML-Seiten lag mir, sodass ich meine Stärken vor allem in diesem Bereich einbringen konnte.

Zu Beginn musste ich mich außerdem erst in GitHub und Visual Studio Code einarbeiten. Ich hatte vorher noch nicht mit diesen Werkzeugen und auch noch nicht an einem so umfangreichen Projekt gearbeitet, in dem die Abstimmung im Team eine wichtige Rolle spielt. Gerade am Anfang musste ich deshalb häufiger nachfragen, wie ich Änderungen hochlade

oder mit Branches arbeite. Dadurch haben sich manche Aufgaben verzögert. Mit der Zeit konnte ich mich jedoch besser orientieren und sicherer mit den Werkzeugen umgehen.

Bei einem ähnlichen Projekt würde ich besonders auf eine frühzeitige und regelmäßige Kommunikation achten. Ich habe zunächst versucht, viele Aufgaben allein zu lösen und dabei auch den zeitlichen Aufwand unterschätzt. Dadurch fehlten teilweise Absprachen über den Bearbeitungsstand und den Zeitplan. In Zukunft würde ich früher nachfragen, wenn Anforderungen oder technische Schritte unklar sind. So lassen sich Missverständnisse schneller klären, und ich kann von den Kenntnissen der anderen Teammitglieder profitieren, statt unnötig lange nach einer Lösung zu suchen.

Für zukünftige Projektgruppen würde ich die Nutzung von GitHub und einer agilen Vorgehensweise empfehlen. Über GitHub konnten wir Aufgaben als Issues festhalten, Teammitgliedern zuordnen und in einem Wochensprint bearbeiten. Durch die Kommentarfunktion und den Status der Issues war nachvollziehbar, wie weit eine Aufgabe fortgeschritten war. Zusätzlich war unser WhatsApp-Chat hilfreich, weil sich dort auch kurze Informationen schnell teilen ließen. Für mich hat das Projekt gezeigt, dass gute Kommunikation eine wichtige Grundlage für die Zusammenarbeit ist – auch bei scheinbar kleinen Fragen oder Änderungen.

Elias Langhans / 7598556
Habe ich strukturiert und organisiert gearbeitet?

Zu Beginn des Projektes habe ich strukturiert gearbeitet. Anfangs habe ich mich an den technischen Tickets auf GitHub orientiert und dabei selbst grobe Pläne erstellt, wann ich etwas erledige. Die zuvor erstellten Dokumente und die definierten APIs haben mir dabei deutlich geholfen. In der mittleren Projektphase bin ich immer mehr von der ursprünglich vereinbarten Struktur abgewichen. Neue Anforderungen wie der Umbau zu GraphQL haben einzelne Features wichtiger gemacht, was dazu führte, dass ich andere Features vernachlässigt habe und später zwischen der Implementierung verschiedener Features gesprungen bin. Diese Punkte haben verursacht, dass mein eigenes strukturiertes Arbeiten und meine Organisation im Laufe des Projektes nachgelassen hat. Im späteren Verlauf erkannte ich selbst, dass die Arbeit so nicht effizient ist, da ich mehrmals nacharbeiten musste und das ständige Wechseln zwischen den verschiedenen Features verhinderte, dass ich mich auf eines konzentrieren konnte. Geholfen hat dabei das Absprechen mit meinen Kommilitonen und dass ich mich wieder an der API Doku sowie die verschiedenen Artefakte der Fallstudie orientiert habe.

Wo sehen Sie Ihre Stärken und Schwächen?

Eine Stärke ist die Lernbereitschaft. Mir war es wichtig, meine eigene Arbeit zu verstehen, statt das Projekt schnell abzuschließen. Deshalb habe ich mich in verschiedene Themen wie z.B. der Integration der externen Buch API oder der Alexa Anbindung eingearbeitet. Außerdem habe ich trotz der Organisationsprobleme die Features eines Sprints bis zum Sprintende fertig gestellt. Eine weitere Stärke ist die Zusammenarbeit. Da wir im Team viele verschiedene Services entwickelten, habe ich eng mit meinen Kommilitonen zusammengearbeitet.
Meine Schwächen lagen vor allem bei der Strukturierung. Im Laufe des Projektes habe ich die API-Doku vernachlässigt und zuvor vereinbarte Punkte nicht identisch umgesetzt. Das führte zu vermeidbarem Mehraufwand. Weil mir das Programmieren mehr liegt als das Dokumentieren, wusste ich später manchmal nicht mehr, warum ich etwas so umgesetzt hatte. Eine Schwäche war auch, dass ich bei Fehlern schnell zu KI griff, statt den Fehler selbst zu suchen. Das führte dazu, dass ich selbst weniger Erfahrung beim Debuggen sammelte. Zudem half KI nicht immer was dazu führte, dass ich manche Fehler selbst beheben musste, nachdem ich bereits viel Zeit in Prompts investiert hatte.

Was würde ich in einem ähnlichen Projekt nun anders machen?

Ich würde mich bei anderen Projekten mehr auf den Aufbau, die Architektur und Dokumentation konzentrieren und Änderungen in der Architektur direkt dokumentieren. Vor allem zu Beginn eines solchen Projekts kann es erheblich erleichtern sich nach diesen

Punkten zu orientieren. Durch meine Erfahrungen im Verlauf des Projektes habe ich erkannt, dass eine durchdachte Architektur und ein ausreichend und logisch definiertes API Design sowie weitere Desing Artefakte die spätere Entwicklung deutlich vereinfachen.
Zusätzlich würde ich die Integration von externen Services früher analysieren. Im Rahmen der Alexa Integration hat sich herausgestellt, dass die Authentifizierung über Account Linking einen eigenen Authentifizierungsmechanismus benötigt. Wäre mir das früher bekannt gewesen hätte die Integration der Alexa vermutlich noch vollständig funktioniert. Zum Ende des Projektes ist das leider nicht mehr möglich, da die Änderung zu komplex wäre.

Was würde ich Studenten des nächsten Jahres empfehlen

Zu empfehlen ist das frühe Anfangen. Als Student hat man am Anfang eines solchen Projektes nicht die Erfahrung, um den tatsächlichen Umfang einzuschätzen. Obwohl wir frühzeitig mit dem Projekt begonnen haben, war es doch mehr Arbeitsaufwand als ich erwartet hätte. Eine weitere Sache ist viel Kommunikation. Durch die Kommunikation mit seinen Kommilitonen können Probleme gemeinsam betrachtet werden und Themen gemeinsam diskutiert werden. Zusätzlich sollte KI-Antworten kritisch betrachtet werden. Gerade bei KI kann es sinnvoll sein sie zu nutzen. Trotzdem muss man die Ergebnisse hinterfragen, da die KI häufig nicht das gesamte Projekt analysiert, sondern nur Teile betrachtet und daraus resultierend falsche und ineffiziente Antworten liefert. Deshalb ist es ratsam KI zu hinterfragen. Darüber hinaus sollten Studenten sich früh mit der Architektur oder der Integration externer Services befassen. Dies kann verhindern, dass externe Services nicht vollumfänglich integriert werden können.

6 Quellenverzeichnis

Devtiro: „The ULTIMATE Guide to Spring Boot: Spring Boot for Beginners“, YouTube, 11.08.2023, https://www.youtube.com/watch?v=Nv2DERaMx-4, abgerufen am 30.10.2026. Genutzt wurden vor allem die Module zu den Schichten einer Spring-App, zur Konfiguration und zu DAOs mit Spring JDBC.

Telusko: „Spring Security 6 with Spring Boot and JWT Tutorial“, YouTube. 11.08.2024, https://www.youtube.com/watch?v=oeni_9g7too, abgerufen am: 30.10.2026. Genutzt wurden alle Module bis Spring Security Google and Github Login.

Microsoft: „Data considerations for microservices“, Azure Architecture Center, https://learn.microsoft.com/en-us/azure/architecture/microservices/design/data-considerations, abgerufen am 30.10.2026.

Dan Vega: „GraphQL for Java Developers“, YouTube. 01.04.2026, https://www.youtube.com/watch?v=xZSv67a9OYA&list=PLZV0a2jwt22u5DEFwYSzOM7mg6AjL90hw abgerufen am: 30.10.2026. Genutzt wurden die Module 01 - 04.